

Animation Lab V2 — Plan d'implémentation

Objectif : Transformer l'Animation Lab en un studio d'animation 2D complet avec édition de sprites, clonage de personnages et gestion d'assets.

Table des matières

- 1. [Phase 1 — Clonage de personnage](#phase-1-clonage-de-personnage)
- 2. [Phase 2 — Éditeur de sprites (style MS Paint)](#phase-2-editeur-de-sprites-style-ms-paint)
- 3. [Phase 3 — Gestion des frames & séquences](#phase-3-gestion-des-frames-sequences)
- 4. [Phase 4 — Intégration & tests](#phase-4-integration-tests)

Phase 1 — Clonage de personnage

1.1 But

Permettre de dupliquer un asset existant (ex: "Champion") → "Champion_v2") pour en créer une variante sans toucher l'original.

1.2 UI — Bouton "Cloner"

- Dans le toolbar de l'Animation Lab, ajouter un bouton **Cloner** à côté du sélecteur d'asset.
- Au clic : popup modale demandant le **nouveau nom** et le **nouveau dossier de destination**.

1.3 Logique de clonage

```
Asset.clone({
  namePrefix : assetsNamePrefix + "_v",
  frameId    : 0,
  totalFrames : 1
})

Clone crée :
- Nouvelle entrée dans AssetCatalog (copie, déplacement, persistance via localStorage)
- Copie du fichier PNG dans public/assets/clones/clones-v2
- Nouvel AssetCatalog (clique pour modifier le clone)
- Mêmes frames, frameId, totalFrames, defaultId, etc.
```

1.4 Persistance

- Stocker les clones dans localStorage sous la clé subcode-asset-clone.
- Au démarrage de l'app, merger ces clones dans ASSET_CATALOG.

1.5 Fichiers touchés

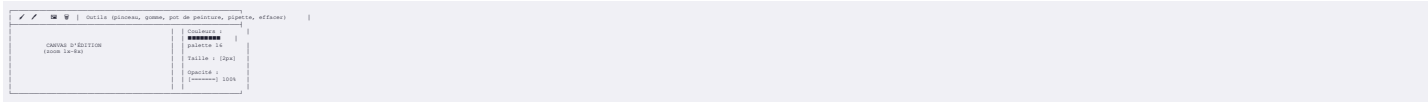
```
- src/renderer-v2/AnimationPageUI.ts → UI bouton + modale
- src/renderer-v2/AssetCatalog.ts → fonction loadClones() + saveClones()
- src/renderer-v2/renderer/Editor-v2.tsx → styles modale + filmstrip clone tag
```

Phase 2 — Éditeur de sprites (style MS Paint)

2.1 But

Offrir un éditeur de dessin intégré pour modifier une frame ou créer une nouvelle image pixel art.

2.2 Layout de l'éditeur



2.3 Outils implémentés

Outil	icône	Comportement
"Pinceau"	✓	Clic + drag = dessin des pixels
"Gomme"	✓	Espace (sans cliquer) ou clic sur le fond
"Pot de peinture"		Flood Fill (remplissage d'une zone de même couleur)
"Pipette"	□	Clic = capture la couleur du pixel sous le curseur
"Formes"	⌘	Rectangle / Cercle plein ou en contour
"Lignes"	⇧	Ligne droite avec shift pour angles multiples de 45°
"Sélection"	⌘	Pixel select + déplacement/couleur la zone

2.4 Canvas édition

- **Backend** : un canvas HTML5 avec image-rendering: pixelated
- **Zoom** : zoom virtuel (affichage zoomé, coordonnées logiques)
- **Grid** : option grille pixel (1px visible au zoom 2-4x)
- **Undo/Redo** : pile d'états (maximum 50 étapes)

2.5 Palette de couleurs

- Palette de 16 couleurs par défaut (SGA style)
- Possibilité d'ajouter une couleur custom via <input type="color">
- Historique des 8 dernières couleurs utilisées

2.6 Export

- Bouton **⌘ Sauver la frame** → remplace la frame courante dans la spritesheet
- Bouton **⌘ Exporter PNG** → télécharge le canvas comme PNG

2.7 Fichiers touchés

- **Nouveau** : src/renderer-v2/SpriteEditorUI.ts → logique complète de l'éditeur
- **Nouveau** : src/renderer-v2/SpriteEditorCanvas.ts → outils (brush, flood fill, shapes, select)
- src/renderer-v2/renderer/Editor-v2.tsx → styles de l'éditeur
- src/renderer-v2/AnimationPageUI.ts → bouton "Éditer" pour chaque frame

Phase 3 — Gestion des frames & séquences

3.1 But

Permettre d'ajouter, supprimer, réordonner des frames dans une animation, et de créer de nouvelles animations.

3.2 Filmstrip amélioré

Drag & drop pour réordonner les frames

Clic droit sur une frame : menu contextuel

→ Insérer une frame vide après

→ Dupliquer cette frame

→ Supprimer cette frame

→ Éditer (ouvre le Sprite Editor)

3.3 Ajout d'une nouvelle image

Processus complet :

1. L'utilisateur clique "Ajouter une image"
2. Modal avec deux options :
 - Importer un fichier : <input type="file" accept="image/png">
 - Créer une frame vide : ouvre le Sprite Editor avec un canvas vide de la taille des frames existantes
3. L'image importée ou créée est ajoutée à la fin de la spritesheet
4. La spritesheet est reconstruite côté client :

```
const canvas = document.getElementById('canvas')
const image = document.getElementById('image')
const ctx = canvas.getContext('2d')
const img = new Image()
img.src = image.src
img.onload = () => {
  // Ajouter une ligne au Data
}
```

5. Les frames de catalogue sont rechargées

6. Le filmstrip est mis à jour

3.4 Nouvelle animation

- Bouton "Nouvelle animation" dans le sélecteur
- Modal : nom de l'animation, frame de début, frame de fin, frameRate, repeat
- La nouvelle animation est ajoutée à la stack list

3.5 Fichiers touchés

- src/renderer-v2/AnimationPageUI.ts → drag & drop, context menu, bouton ajout
- **Nouveau** : src/renderer-v2/SpriteEditorBuilder.ts → reconstruction de spritesheet côté client
- src/renderer-v2/renderer/Editor-v2.tsx → drag ghost, context menu

Phase 4 — Intégration & tests

4.1 Architecture finale

```
AnimationPageUI
├── SpriteEditorUI (modal overlay)
├── SpriteEditorCanvas (brush, flood fill, shapes, select)
├── Filmstrip (frames + drag & drop + context menu)
├── SpriteEditorBuilder (reconstruction client-side)
└── AssetCatalog (liste persistance localStorage)
```

4.2 Tests

1. **Build** : npm run build → 0 erreur
2. **TypeScript** : npm run -c build → clean
3. **Manuels**
 - Cloner un personnage → apparaît dans le sélecteur
 - Éditer une frame → les changements persistent
 - Ajouter une image → nouvelle frame visible dans le filmstrip
 - Drag & drop frames → réordonnement effectif

4.3 Livrables

Fichier	Description
src/renderer-v2/AnimationPageUI.ts	Page principale (modifiée)
src/renderer-v2/SpriteEditorUI.ts	Éditeur de sprites (nouveau)
src/renderer-v2/SpriteEditorCanvas.ts	Outils de dessin (nouveau)
src/renderer-v2/SpriteEditorBuilder.ts	Builder de spritesheet (nouveau)
src/renderer-v2/AssetCatalog.ts	Clones + persistance (modifié)
src/renderer-v2/renderer/Editor-v2.tsx	Styles (modifié)

Estimation

Phase	Durée estimée
Phase 1 — Clonage	2h
Phase 2 — Éditeur MS Paint	8h
Phase 3 — Gestion frames	4h
Phase 4 — Intégration	2h
"Total"	~16h"